National University of Computer and Emerging Sciences
Islamabad Operating System Lab Spring 2025

Lab Task:Process Creation using Fork, Wait, and Exit System Calls

---

Fork Visualizer for Practice

Before diving into the tasks, it's beneficial to familiarize yourself with the `fork()` system call and how processes are created. You can practice and visualize the process creation flow using the following online Fork Visualizer:

Online Fork Visualizer

This tool allows you to simulate the behavior of the `fork()` system call in a graphical way, helping you better understand how parent and child processes are created and interact.

Task 1: Add and Subtract

Objective:
Write a program where the **child process** calculates the sum of two numbers, and the **parent process** calculates the difference of the same numbers after the child process has completed its work. The parent will not use the child's result, but simply perform its own calculation.

Input:

- Accept two integers from the user.

Child Process:

- The child process will calculate the sum of the two numbers and print it.

Parent Process:

- After the child process completes, the parent process will calculate the difference between the two numbers and print the result.

Expected Input/Output:
Example 1:

Input:

```
Enter the first number: 8

Enter the second number: 3
```

-

Output:

```
Child result (sum): 11

Parent result (difference): 5
```

-

Task 2: Check Even or Odd

Objective:
Write a program where the **child process** checks if a given number is even or odd, then the **parent process** prints the result after the child has completed its task.

Input:

- Accept an integer from the user.

Child Process:

- The child process will check whether the number is even or odd and print the result.

Parent Process:

- After the child process finishes, the parent will print whether the number is even or odd.

Expected Input/Output:
Example 1:

Input:

```
Enter a number: 4
```

-

Output:

```
Child result: even

Parent result: even
```

-

Example 2:

Input:

```
Enter a number: 7
```

-

Output:

```
Child result: odd

Parent result: odd
```

-

Task 3: Simple Child Processes for Operations

Objective:
Create a program where the **parent process** creates child processes to perform three arithmetic operations (addition, subtraction, and multiplication). The child processes perform their operations first, and the parent waits for all children to finish before printing the results.

Steps:

1. **Parent Process:**
    - Accept two integers `a` and `b` from the user.

    - Create 3 child processes using `fork()`.
    - The first child performs addition, the second performs subtraction, and the third performs multiplication.
    - The parent process waits for all child processes to finish and then prints the results.
2. **Child Processes:**
    - The first child performs the addition of `a` and `b` and prints the result.
    - The second child performs the subtraction of `a` and `b` and prints the result.
    - The third child performs the multiplication of `a` and `b` and prints the result.

Expected Input/Output:
Example 1:

Input:

```
Enter the first number: 5

Enter the second number: 3
```

-

Output:

```
Child 1 result (addition): 8

Child 2 result (subtraction): 2

Child 3 result (multiplication): 15
```

- `Parent result: All child processes completed.`

Deliverables:

- **Submit the c code for each of the tasks separately (in .cpp files).**
- **Provide screenshots or output showing the results of running the programs**

THE END